```
np.random.randint(0,mnist_data.train_data.shape[0],size=batch_size)
    x = mnist_data.train_data[rid].float().flatten(start_dim=1)
    x /= x.max()
    pred = nn(x,w1,w2)
    target = mnist_data.train_labels[rid]
    loss = lossfn(pred,target)
    loss.backward()                                    3
    optim.step()                                       4
    optim.zero_grad()                                  5
```

- ***1*** Sets up the loss function
- ***2*** Sets up the ADAM optimizer
- ***3*** Backpropagates
- ***4*** Updates the parameters
- ***5*** Resets the gradients

You can see that the loss function in figure A.4 is much smoother now with the Adam optimizer, and it dramatically increases the accuracy of our neural network classifier.

Figure A.4. The loss plot of our neural network trained on MNIST with the built-in PyTorch optimizer Adam.

